

Verification — Work Order Details: paginate all content sections

Ticket: “[Web] Work Order Details — Paginate all content sections (Assets, Issues, Tasks, Photos, Attachments)” · Environment: QA **acme.qa.egalvanic.ai** (app badge V1.36) · Page verified: `/sessions/{id}` (Work Order Details) · Verified: **2026-07-31**, authenticated Super-Admin session

✅ **IMPLEMENTED — all five named sections are server-side paginated with controls and total counts.**

Each section lazily calls a versioned `/v2` endpoint with `limit + offset`, renders a “Rows per page / 1–N of N” footer, and shows a total count in both the tab badge and the section heading (e.g. “IR Photos (8)”). One residual gap: the **Forms** sub-tab is still unpaginated.

1. Per-section result (live network capture)

Section	Endpoint actually called	Page size	Pagination UI	Total count shown	Verdict
Assets	<code>GET /ir_session/{id}/assets/v2?limit=20&offset=0</code>	20	Rows-per-page + range footer	Tab badge “Assets 1”	PAGINATED
Tasks	<code>GET /ir_session/{id}/tasks/v2?limit=20&offset=0</code>	20	Rows-per-page + range footer	Tab badge	PAGINATED
Issues	<code>GET /ir_session/{id}/issues/v2?limit=20&offset=0</code>	20	Rows-per-page + range footer	Tab badge	PAGINATED
IR Photos	<code>GET /ir_session/{id}/photos/v2?limit=25&offset=0</code>	25	Rows-per-page (10/25/50) + “1–8 of 8”	Badge “8” + heading “IR Photos (8)”	PAGINATED
Attachments	<code>GET /ir_session/{id}/attachments/v2?limit=25&offset=0</code>	25	Rows-per-page + range footer	Tab badge	PAGINATED
Forms	<code>GET /eg-form-instance/by-session/{id} — no params</code>	—	grid footer only (client-side, “0–0 of 0”)	no badge	NOT SERVER-PAGINATED

Each section fetches **lazily on tab activation** (verified by diffing `performance.getEntriesByType('resource')` before/after each tab click) — so opening a work order no longer downloads every section's rows up front. Additionally a `GET /ir_session/{id}/summary/v2` supplies the per-section totals (`counts: {assets, issues, tasks, photos, attachments, asset_nodes}`) without fetching rows.

2. Proof the pagination is server-side (not client-side slicing)

Two independent checks — this is the distinction the ticket asks for (“Backend APIs support limit/offset ... for each section”):

Check	Observation
Changing “Rows per page” issues a NEW request	Setting IR Photos to 10 → fresh call <code>photos/v2?limit=10&offset=0</code> (a client-side grid would re-slice in memory with no network call).

Check	Observation
Distinct offsets return distinct records + metadata	photos/v2?limit=3&offset=0 → 3 rows, first id d4acb762..., pagination {limit:3, offset:0, total:8, has_more:true} photos/v2?limit=3&offset=3 → 3 rows, first id e62f6926..., pagination {limit:3, offset:3, total:8, has_more:true}

3. Acceptance criteria

Criterion	Status
Each section loads a reasonable page size (e.g. 20–25) with pagination controls or infinite scroll	MET — 20 (Assets/Tasks/Issues), 25 (Photos/Attachments)
Backend APIs support limit/offset or cursor pagination for each section	MET — limit / offset + total / has_more ; Forms excepted
Page load time significantly improved for work orders with 50+ items	NOT VOLUME-TESTED — mechanism verified; QA tenant has no 50+ item work order (largest available: 8 photos). Needs seeded data or a Prod-scale WO to measure.
Total count displayed per section (e.g. “Assets (147)”)	MET — tab badges + section heading “IR Photos (8)”

4. Screenshots

 **Egalvanic**

DASHBOARDS

Site Overview

Sales Overview

Ops Overview

SITE DATA

Arc Flash Readiness

Condition Assessment

Panel Schedules

Equipment Designations

Work Orders

Role: Super Admin

abhiyant admin

Job - Jul 31, 3:30 PM

General

Assets 1

Tasks

Forms

Issues

IR Photos 8

Attachments

IR Photos (8)

Upload IR Photos

IR Photo	Visual Photo	Linked Issues	Asset Name	Location	Associated Issue	Captured
			cb 2	b1 > floor 1 > room1	-	Jul 31, 2026 2

Rows per page: 10 1–8 of 8

Close Work Order

IR Photos — pagination + total count. Tab badges show per-section totals (“Assets 1”, “IR Photos 8”); the section heading reads “**IR Photos (8)**” — exactly the format the ticket’s AC #4 requests. The grid footer provides “**Rows per page: 10**” with a 10/25/50 selector and the range “**1–8 of 8**” plus prev/next controls. Selecting a different page size re-queries the server (photos/v2?limit=10&offset=0).

Assets — paginated grid. Populated by `assets/v2?limit=20&offset=0` (page size 20) with its own rows-per-page/range footer, and the location tree on the left for scoping. Tasks and Issues follow the identical pattern at `tasks/v2` and `issues/v2`.

Forms — the residual gap. This tab calls `GET /eg-form-instance/by-session/{id}` with **no limit/offset**, and the response carries no `pagination` object (confirmed by re-requesting with `?limit=1&offset=0` — the parameters are ignored). The grid does show a footer (“0–0 of 0” here, since this work order has no forms), but that is **client-side** paging over a fully-downloaded array — the server always returns every form instance. Low impact today (form instances per work order are few), but it is the one section still matching the ticket’s original “loads all records at once” description, and is covered by the ticket’s “Any other sub-sections (Forms, Connections, etc.)”.

5. Recommendation

- **Close the main scope** — Assets, Issues, Tasks, Photos and Attachments all meet the pagination, controls and total-count criteria with genuine server-side `limit / offset`.
- **Keep open (or split out) two items:** (a) paginate the **Forms** sub-tab for parity; (b) run the **50+ item load-time measurement** once a large work order exists on QA — the improvement is architecturally implied by lazy per-tab fetching but has not been measured against a numeric target.

Method. A real work order was opened at `/sessions/b6bcc81d-...` and each of the six tabs activated in turn; the API calls attributable to each tab were isolated by diffing the browser’s resource-timing buffer around every click. Server-side behaviour was then confirmed directly against the API from the same authenticated session. Note: the repo’s frontend reference clone (snapshot of `developer @ d168200`, 30 May 2026) still shows the pre-fix single `/full` load-everything pattern and no `/v2` endpoints — that clone is stale; all findings above are from the live QA build.